

chapter8

제8장 피처 스토어 구축

7장까지 데이터를 수집하고(4장), 중복을 제거하고(5장), 실시간으로 집계하고(6장), 배치로 정제해 확정했다(7장). 이제 처음으로 그 데이터의 소비자가 모델이 되는 지점에 도착했다. 모델은 원천 데이터를 그대로 입력으로 받지 않는다 — “지역구별 전일 민원 건수” 같은 가공된 입력, 즉 피처(feature)를 받는다. 그런데 피처에는 까다로운 요구가 하나 있다: 훈련할 때 본 값과 예측할 때 보는 값이 같은 방식으로 계산된 값이어야 한다. 훈련은 과거 데이터 위에서 배치로, 예측은 지금 이 순간 실시간으로 일어나므로, 이 요구는 저절로 충족되지 않는다. 이 장은 그 요구를 구조로 해결하는 피처 스토어를 다룬다. 피처의 정의·저장·조회를 한곳으로 모으는 발상을 이해하고, 오프라인·온라인 이중 저장 구조와 미래 누출을 막는 point-in-time 조회를 Feast 최소 구성으로 직접 실행하며 관찰한다. 실습(★★★☆☆)의 피처 원천은 7장 배치가 확정한 일별 집계 그대로다 — 7장의 예고 (“일별 집계가 오프라인 피처의 재료가 된다”)가 이 장에서 실현된다.

학습 목표

피처 스토어가 어떤 문제(훈련-서빙 불일치·중복 구현)를 구조로 해결하는지 설명할 수 있다.
오프라인·온라인 store의 소비 패턴 차이와 materialize의 역할을 실측으로 확인할 수 있다.
point-in-time correctness(미래 누출 방지)를 실제 조인 결과로 검증하고, 피처 정의서를 작성할 수 있다.

요구사항에서 피처 설계표(entity 키·TTL·집계 창·materialize 주기·feature service 묶음)를 도출하고, 각 값의 근거와 그 설계가 깨지는 조건을 적을 수 있다.

8.1 피처 스토어의 역할

학습 목표

“피처를 각자 계산”하는 구조가 왜 사고로 이어지는지 이해한다.
피처 스토어의 세 기능(정의·저장·조회의 일원화)을 구분한다.

직관: 같은 숫자를 두 곳에서 두 번 만들면 생기는 일

“지역구별 전일 민원 건수”를 입력으로 쓰는 민원 급증 예측 모델을 가정한다. 피처 스토어가 없다면 이 수치는 두 곳에서 따로 산출된다. 훈련 파이프라인은 과거 데이터 위에서 pandas로 집계하고, 서빙 API는 요청이 올 때마다 운영 DB에 SQL을 실행한다. 두 코드는 같은 것을 계산하려는 서로 다른 구현이고, 서로 다른 구현은 반드시 어긋난다 — 한쪽은 중복 제거를 했고 다른 쪽은 안 했다면

(5장), 한쪽은 미매핑을 뺀고 다른 쪽은 넣었다면(7장), 모델은 훈련 때 본 적 없는 분포의 입력을 서버에서 받는다. 이것이 **훈련-서빙 스큐(training-serving skew)**이며, 성능 저하가 원인 불명으로 나타나는 고전적 사고 유형이다.

이 실패는 이 분야에서 이름이 붙은 일급 위험이다. Google의 실무 규칙 문서 “Rules of ML”은 훈련-서빙 스큐를 “훈련 시점의 성능과 서빙 시점의 성능 사이의 차이”로 정의하고 반복 발생하는 대표 실패로 다룬다(방법론 차용 — 민간·일반 대상, 1장이 이미 인용한 출처다). 피쳐 스토어를 만든 실무자들의 진단은 더 직설적이다 — 피쳐 계산이 훈련과 서빙에서 어긋나면 “치명적이고 디버깅하기 어려운(catastrophic and hard-to-debug)” 성능 문제가 생긴다고 벤더 기술 문서는 적는다(Tecton, 방법론 차용). 용어를 통일한다: 1장은 같은 현상을 **학습-서빙 불일치**로 불렀다 — 훈련=학습, 서빙 시점의 값 어긋남이라는 실체는 같고, 이 장은 피쳐 계산의 관점에서 “스큐”라 부른다.

피쳐 스토어의 발상은 계산을 두 번 하지 않는 것이다: 피쳐를 **한 번 정의하고, 한 곳에 저장하고, 훈련과 서빙이 같은 곳에서 조회한다**. 구체적으로 세 가지가 일원화된다.

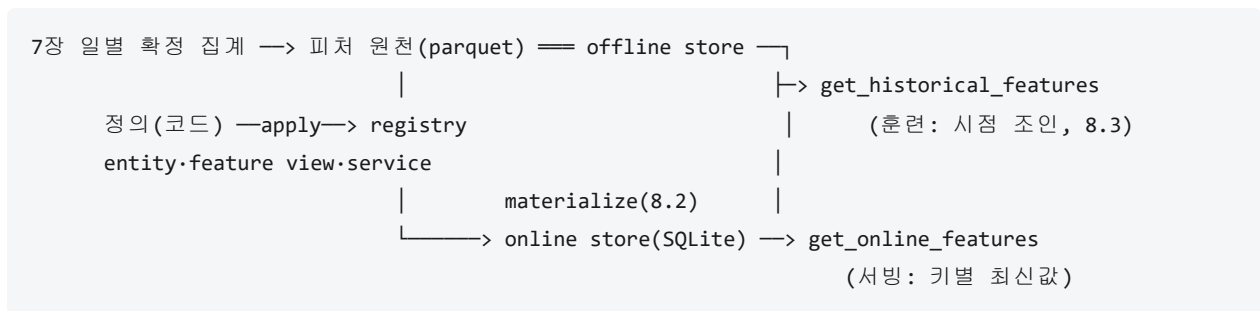
정의의 일원화: “이 피쳐는 어떤 원천에서 어떤 스키마로 계산되는가”를 코드로 선언하고 버전 관리한다. 실습에서는 entity(조인 키)와 feature view(피쳐 묶음)의 선언이 이것이다.

저장의 일원화: 피쳐 값은 피쳐 스토어의 저장소에 있다 — 훈련용 과거 이력 전체(오프라인)와 서빙용 키별 최신값(온라인)의 이중 구조다(8.2).

조회의 일원화: 훈련은 `get_historical_features` (시점 기준 조인), 서빙은 `get_online_features` (키 기반 최신값) — 서로 다른 API지만 **같은 정의, 같은 값을 읽는다**.

7장과의 연결이 이 장의 핵심 구조다. 7장 배치는 정제·검증된 “지역구별 일별 확정 집계”를 만들었다 — 그것이 바로 피쳐의 원천이다. 피쳐 스토어는 새 계산을 발명하는 곳이 아니라, **이미 확정된 배치 산출물을 모델이 소비 가능한 계약으로 바꾸는 곳**이다. 배치 파이프라인(생산자)과 모델(소비자) 사이의 창구라고 보면 정확하다.

실습 구성 전체를 다이어그램으로 제시한다 — 이 다이어그램의 각 상자를 이 장의 절들이 하나씩 채운다.



피쳐 스토어는 데이터베이스와 무엇이 다른가

“결국 저장소 아닌가”라는 의문에 답하면 다음과 같다. 데이터베이스는 값을 저장하지만, 그 값이 **어떻게 계산되었고 언제 알려졌으며 누가 책임지는지**는 기록하지 않는다. 피쳐 스토어의 추가분은 정확히 그 메타 계층이다: 정의가 코드로 registry에 있고(감사 가능), 조회가 시점 규율을 강제하고

(8.3), 정의서가 소유·품질 책임을 명시한다(8.4). 반대로 피처 스토어는 범용 질의 엔진이 아니다 — “키와 시점을 주면 그때의 피처를 반환한다”는 좁은 계약에 특화된 창구다. 저장 기술 자체는 평범하다(실습은 parquet와 SQLite다) — 가치는 저장기 아니라 **규율**에 있다.

이 좁은 계약을 현장은 한 문장으로 요약한다 — 피처 스토어는 “모델과 데이터 사이의 인터페이스 (the interface between models and data)”다(Tecton, 방법론 차용). 이 장이 피처 스토어를 “확정된 배치 산출물을 모델이 소비 가능한 계약으로 바꾸는 창구”(8.1 앞부분)로 본 것과 정확히 같은 자리다. 이 개념의 출발점도 같은 문서가 설명한다 — Tecton 문서는 피처 스토어라는 개념이 Uber의 ML 플랫폼(Michelangelo)을 소개한 글에서 처음 등장했다고 설명한다(원 블로그는 접근이 제한되어 Tecton 경유로 인용 — 배경 자료). 이후 피처 스토어는 여러 오픈소스·상용 구현으로 퍼졌고, 실습이 쓰는 Feast도 그중 하나다. 요점은 도구 이름이 아니라, 훈련·서빙 스큐라는 특정 실패를 겨냥해 만들어진 특화 계층이라는 계보다.

워크드 예제: 스큐는 어떻게 생기는가 — 7장 실측으로

“각자 계산”이 실제로 어떻게 어긋나는지, 7장의 7월 2일 실측으로 확인한다. 그날의 물리 레코드는 18건, 중복 제거 후 유효 17건, 표준 코드 매핑 16건이었다(7장 표의 실측). 이제 세 팀이 각자 “7/2 민원 건수” 피처를 구현한다면:

구현	계산 방식	“7/2 전체 건수”
훈련 파이프라인	7장 확정 집계(매핑 완료분) 사용	16
서빙 API	운영 DB 원본을 그대로 COUNT	18
분석 부서	중복만 제거하고 미매핑 포함	17

세 값 모두 “그날 민원 건수”라는 같은 이름을 사용하지만 셋 다 다르다 — 어느 것도 버그가 아니고, 정제 단계 중 어디까지를 적용했는가의 차이일 뿐이다. 모델이 16을 기준으로 훈련되고 18을 기준으로 서빙되면, 입력 분포가 체계적으로 어긋난다. 피처 스토어가 강제하는 것은 셋 중 무엇이 옳으냐의 판정이 아니라, **하나를 정의로 선택하고 모두가 그것을 사용하게 하는 것이다**.

표 8.1 피처 스토어 유무 비교

항목	각자 계산(스토어 없음)	피처 스토어
피처 정의	훈련 코드와 서빙 코드에 각각 존재	한 곳에 선언, 버전 관리
계산 일치	구현이 다르면 어긋남(스큐)	같은 값을 두 경로로 조회
재사용	팀·모델마다 재구현	정의 공유, 소유자 명시(8.4)
미래 누출 방지	훈련 코드 작성자의 주위에 의존	point-in-time 조회가 구조로 보장(8.3)
책임 소재	불명(누구의 집계가 맞나)	피처 정의서의 소유자·원천 명시

언제 불필요한가 — 반대 논거

피처 스토어가 항상 답인 것은 아니다. 모델이 하나뿐이고 배치 추론만 하며 피처 계산이 훈련 파이프라인 안에 완결되어 있다면, 저장소를 하나 더 추가하는 것은 관리 대상만 늘린다 — 훈련·서빙이 같은 배치 코드에서 나오면 스큐 자체가 성립하지 않기 때문이다. 이중 구조가 필요한 조건은 대체로 셋 중 하나다: 온라인 서빙이 있어 두 소비 경로가 실재하거나, 여러 모델·팀이 같은 피처를 재사용하거나, 미래 누출 감사가 요구되는 도메인이거나. 실습이 최소 구성(파일+SQLite)인 이유도 같은 맥락이다 — 구조의 규율을 배우는 데 무거운 인프라는 필요 없다.

공공 사례

북지 사각지대 발굴 모델이 “가구별 최근 3개월 공과금 체납 횟수”를 피처로 사용한다고 가정한다. 훈련 팀은 통계 부서의 월간 확정 파일로 집계했고, 서빙 시스템은 운영 DB의 실시간 테이블을 읽었다면 — 같은 이름의 피처가 사실은 다른 수치다(확정 파일에는 소급 정정이 반영되고, 운영 테이블에는 아직이다). 모델 성능 저하의 원인을 몇 주간 모델의 문제로 추적하다가 피처 계산 차이로 밝혀지는 일이 실제로 흔하다. 검수 관점의 질문: 훈련과 서빙이 **같은 피처 정의**를 참조하는가, 아니면 “같다고 믿어지는” 두 구현이 있는가. ### 핵심 정리 - 피처를 훈련·서빙이 각자 계산하면 구현 차이가 훈련·서빙 스큐가 된다. - 피처 스토어는 정의·저장·조회를 일원화해 “같은 값”을 구조로 보장한다. - 피처의 원천은 새 계산이 아니라 7장까지 만든 확정 산출물이다.

8.2 offline store와 online store

학습 목표

같은 피처의 두 소비 패턴(훈련 vs 서빙)이 왜 다른 저장소를 요구하는지 이해한다.
materialize(오프라인→온라인 적재)의 역할과 적재 누락 위험을 실측으로 확인한다.

직관: 이력 전체를 보는 소비와 최신값만 보는 소비

훈련과 서빙은 같은 피처를 전혀 다르게 소비한다. 훈련은 “지난 석 달분, 모든 지역구, 모든 날짜”를 한 번에 읽는 대량·과거 이력·지연 허용 방식이고, 서빙은 “지금, 강남구, 최신값 하나”를 밀리초 안에 읽는 소량·최신값·저지연 방식이다. 하나의 저장소로 두 패턴을 다 잘하기는 어렵다. 그래서 피처 스토어는 저장을 둘로 나눈다.

offline store: 피처의 전체 이력을 축적하는 곳. 실습에서는 parquet 파일이며, 운영에서는 데이터 웨어하우스가 흔하다. 훈련용 시점 조인(8.3)이 여기서 일어난다. 전체 이력을 보존하며, 조회 지연은 허용된다.

online store: 키별 **최신값만** 두는 곳. 실습에서는 SQLite이며, 운영에서는 Redis 같은 키-값 저장소가 흔하다. 서빙 API가 여기서 읽는다. 최신값만 즉시 제공한다.

materialize: 오프라인의 최신값을 온라인으로 옮기는 배치 작업. 7장의 DAG에 태스크 하나로 들어갈 성격의 작업이다 — “일별 집계 확정 → 피처 원천 갱신 → materialize”가 자연스러운 하루 주기다.

두 store의 성격 차이를 실습 구성 그대로 정리하면 다음과 같다.

표 8.2a offline store와 online store(실습 구성 기준)

항목	offline store	online store
담는 것	피처의 전체 이력(실습: 9행 = 3일 × 3지역)	키별 최신값만(실습: 3행)
소비자 / 패턴	훈련 — 대량 일괄, 시점 조인	서빙 — 키 하나, 최신값, 저지연
실습 구현	parquet 파일	SQLite
운영에서 흔한 구현	데이터 웨어하우스·데이터 레이크	Redis 등 키-값 저장소
채워지는 방식	배치 산출물 적재(7장의 출력)	materialize(오프라인에서 복사)
지연·시간 특성	갱신은 배치 주기, 조회는 느려도 됨	조회 밀리초, 최신성은 materialize 주기

마지막 행이 6·7장의 절충 구조와 정확히 평행하다: 온라인 피처의 적시성 한계는 materialize 주기이며, 이것은 배치 지연(7.1)이 피처 계층에 그대로 상속된 것이다. 이는 실습 구성의 우연이 아니라 현장의 통례다 — 온라인 스토어는 대개 최신 상태를 즉시가 아니라 궁극적으로 맞추는 (eventually consistent) 저장소이고, 피처의 최신성 요구에 맞춰 적재 주기를 정한다 (Tecton·Hopsworks, 배경 자료). “실시간 피처가 필요하다”는 요구는 곧 materialize를 스트리밍화 (6장)해야 한다는 뜻이고, 그만큼 비용이 추가된다 — 일 1회 확정 피처로 충분한 문제인지가 설계의 첫 질문이다.

materialize 주기는 무엇에서 도출하는가

주기를 정하는 순서는 거꾸로다. 먼저 모델이 견딜 수 있는 피처의 낡음 정도를 정하고, 그다음 적재 주기를 그 안쪽으로 잡는다. 낡음의 정도는 **피처 최신성(feature freshness)** 으로 측정한다 — 온라인 스토어에 들어 있는 값의 타임스탬프와 현재 시각의 차이다. 하루 한 번 적재하면 이 값의 최댓값은 약 24시간이 된다.

24시간이 허용 가능한지는 두 가지가 정한다. 피처가 가리키는 현상이 얼마나 빨리 변하는가, 그리고 그 값으로 내리는 결정을 되돌리는 비용이 얼마인가. “전일 민원 건수”로 다음 날 인력 배치를 정한다면 하루 지연은 정의상 문제가 아니다 — 피처 자체가 전일 값이고, 배치는 다음 날 다시 조정할 수 있다. 반대로 재난 신고 밀집도로 출동 순서를 정한다면 24시간 전 값은 결정에 쓸 수 없고, 잘못 내린 순서를 되돌릴 방법도 없다. 이때 선택지는 둘이다: materialize를 스트리밍으로 바꾸거나(6장 — 운영 비용과 장애 지점이 함께 늘어난다), 최신성 요구가 큰 피처만 별도 view로 분리해 짧은 주기로 실행하고 나머지는 일 배치로 남기거나. 민간에서 이 최신성은 SLA(서비스 수준 약정) 항목으로 계약서에 들어가고, 공공에서는 “이 결정이 몇 시간 전 값으로 내려졌는가”라는 감사 질문의 답이 된다.

워크드 예제: 실습의 이중 구조 실측

같은 값이 두 곳에 있으므로, 두 곳이 정말 같은지가 곧 품질 항목이 된다. 실습은 세 지역구의 오프라인 최신값과 온라인 조회값을 기계적으로 대조해 **훈련-서빙 피쳐 일관성 점검표**를 만들었다 — 표 8.2b가 그 실측이다.

실습 8.1의 피쳐 원천은 7장 확정 집계 9행(3일 × 3개 지역구)이다. 이 9행이 모두 들어 있는 곳이 offline store(parquet)이고, materialize 후 online store(SQLite)에는 지역구당 최신 1행씩 3행만 남는다 — 7/3 집계(강남 9·마포 6·관악 5)다. 온라인 조회와 오프라인 최신값의 대조 결과는 다음과 같다(ch7_consistency_report.json, 실측).

표 8.2b 훈련-서빙 피쳐 일관성 점검표(실제 실행)

지역구(코드)	offline 최신값(건수·미매핑율)	online 조회값(건수·미매핑율)	일치
마포구(11440)	6 · 0.045	6 · 0.045	○
관악구(11620)	5 · 0.045	5 · 0.045	○
강남구(11680)	9 · 0.045	9 · 0.045	○

세 키 모두 일치(all_consistent=true)했다. 값 자체가 7장 실측(7/3 일별 집계: 강남 9·마포 6·관악 5, 미매핑율 0.045)과 같다는 점도 확인한다 — 배치 산출물이 변형 없이 피쳐가 되었다는 뜻이다. 이 표가 자명해 보인다면 그것이 요점이다: 이 자명함은 “두 저장소를 같은 정의에서 적재”했기 때문이지 저절로 오는 것이 아니며, 각자 계산 구조에서는 이 표의 어느 행이든 어긋날 수 있다.

이 대조를 하는 이유는 도메인을 가리지 않는다. 민간에서도 오프라인·온라인 피쳐값을 정기적으로 대조하고 차이가 나면 알림을 발생시킨다 — 추천 피쳐가 어긋나면 클릭률이 저하되고 그것이 곧바로 매출에 반영되기 때문이다. 다른 것은 점검표가 남는 위치다. 공공에서는 감사 증빙 문서가 되고, 민간에서는 운영 대시보드의 한 지표로 관리된다. 다만 저장 값이 같다고 안심할 수는 없다. 두 저장소가 같은 값을 갖고 있어도 서빙 코드가 조회 뒤에 그 값을 다르게 변형하면 모델이 받는 입력은 여전히 어긋난다 — 그래서 운영에서는 서빙 시점에 모델로 들어간 값을 로그로 남겨 훈련 분포와 정기 비교한다(보충 참조).

실제로 발생한 문제: 값이 없는 온라인 스토어

이 실습을 구성하며 실제로 발생한 오류를 여기에 기록하는 이유는, 이것이 8장 버전의 침묵 실패이기 때문이다. 처음에는 적재를 materialize_incremental (증분 적재)로 호출했는데, 이 함수의 기본 시작점은 “현재 시각 – TTL”이다. 실습 피쳐의 타임스탬프(7/2~7/4)는 그 창보다 과거라 **아무것도 적재되지 않았고**, 에러도 없었다 — 온라인 조회가 조용히 None을 반환할 뿐이었다. 4장의 유실, 6장의 폐기, 7장의 정제 drop에 이어 또 한 번: **비어 있는 결과는 별도 확인 없이는 발견되지 않는다**. 수정은 적재 범위를 명시하는 것이었고(materialize(start_date=..., end_date=...)), 교훈은 일반적이다 — 적재 작업의 성공 로그가 아니라 **적재된 건수**를 확인해야 하며, 서빙 전에 표 8.2b 같은 일관성 대조를 자동화해야 한다.

온라인 조회 실패를 상시 관측하는 지표가 **피쳐 결측률(feature null rate)** 이다 — 서빙 조회가 값을 찾지 못하고 None을 돌려준 비율이다. 이 비율이 갑자기 상승하면 materialize가 실패했거나 기준 시점이 TTL을 넘겼다고 의심한다. 결측을 0 같은 기본값으로 대체하면 지표 자체가 소실되므로, 대체하지 않고 계수하는 쪽을 택한다(11장).

핵심 코드

두 소비 경로의 API가 다르다는 것이 이중 구조의 요약이다.

```
train_df = store.get_historical_features(entity_df, features=svc).to_df() # 훈련: 시점 조인
store.materialize(start_date=t0, end_date=now) # 적재(범위 명시)
serving = store.get_online_features(features=svc,
                                     entity_rows=[{"lawd_cd": "11680"}]) # 서빙: 최신값
```

전체 코드는 *practice/chapter7/code/7-1-feature-store-minimal.py* 참고

공공 사례

재난 대응 우선순위 모델을 가정한다. 훈련은 분기마다 — 지난 2년의 지역별 피쳐 이력 전체를 offline store에서 읽는다. 서빙은 신고가 들어올 때마다 — 해당 지역구의 최신 피쳐를 온라인에서 밀리초에 읽는다. 만약 서빙이 훈련처럼 웨어하우스에 질의한다면 응답 시간이 초 단위로 증가해 상황실 시스템이 사용할 수 없고, 훈련이 서빙 저장소처럼 최신값만 있는 곳에서 읽는다면 과거 이력이 없어 시점 조인(8.3) 자체가 불가능하다. 이중화는 사치가 아니라 두 소비 패턴의 물리적 요구가 다른 데서 온 필연이다 — 검수 질문은 “두 저장소의 값이 같음을 무엇으로 보증하는가”(표 8.2b의 자동화 여부)다.

민간 실시간 피쳐 공급의 공공 적용

온라인 스토어가 존재하는 이유 — 서빙이 최신 피쳐를 밀리초에 읽어야 한다는 요구 — 는 민간에서 가장 먼저, 가장 집중적으로 다뤄졌다. 추천 시스템은 이용자가 방금 누른 상품을 다음 화면의 입력 피쳐로 즉시 반영해야 하고, 카드 사기 탐지는 지금 이 거래의 직전 몇 초 행동을 피쳐로 받아 승인 또는 차단을 결정한다. 두 경우 모두 “훈련은 과거 배치, 서빙은 지금 이 키의 최신값”이라는 이 장의 이중 구조가 그대로 작동하며, 실시간 피쳐 공급이 곧 서비스 품질이 된다. 공공에서 실시간 피쳐가 덜 다뤄져 온 것은 데이터가 실시간이 아니어서가 아니라(재난·민원은 본질적으로 실시간이다) 운영 관행이 연·분기 보고에 맞춰져 있었기 때문이다. 민간의 방법을 공공 문제에 적용하면 다음과 같다.

표 8.2c 민간 실시간 피쳐 방법의 공공 적용

민간 현장 방법	무엇을 하는가	공공 대응물	공공 특수성에 따른 변형
추천 시스템 실시간 피쳐	이용자 최근 행동으로 모델 입력을 즉시 갱신	민원 배정 우선순위 피쳐 — 접수 급증·처리 지연을 실시간 반영해 배정 순서 산정	피쳐의 정의·집계 기준을 문서화하고 배정 근거를 설명할 수 있어야 함 (자동 결정 설명 책임, 1.4·16장)
사기 탐지 실시간 피쳐	직전 수 초의 행동을 피쳐로 즉시 판정	재난 긴급도 피쳐 — 신고 밀집도·유형을 실시간 집계해 대응 우선순위 산정	오탐·미탐의 비용이 비대칭(놓친 재난 vs 헛출동) — 임계와 사람 개입 지점을 명시
배치 피쳐의 정기 갱신	일·시간 단위 확정 피쳐를 materialize	복지 사각지대 발굴 피쳐 — 공과금 체납·서비스 이용 이력을 주기 집계	개인·가구 단위라 접근 등급·이용 목적 제한이 불음(표 8.5의 3등급, 13장)

세 사례에 공통된 공공 특수성이 하나 있다. 민간에서 실시간 피쳐의 품질 기준은 대체로 “정확하고 빠른가”에서 멈추지만, 공공에서는 여기에 **출처와 시점의 증명**이 더해진다 — 이 피쳐가 어느 원천에서, 언제 알려진 값으로 계산되었는가를 사후에 감사가 요구한다. 그래서 이 장의 두 규율(피쳐 정의서의 출처·소유자 명시, 타임스탬프의 “알려진 시점” 의미)은 공공에서 선택이 아니라 감사 대응의 골격이 되고, 13장의 피쳐 거버넌스로 확장된다. 공공에서는 민간의 실시간 피쳐 공급 기법에 출처와 시점의 증명 요건을 추가한다. ### 핵심 정리 - 훈련(대량·이력)과 서빙(소량·최신·저지연)의 소비 패턴이 달라 저장소를 이중화한다. - materialize는 오프라인 최신값의 온라인 적재이며, 적재 범위·건수를 확인하지 않으면 조용히 빈다. - 훈련-서빙 일관성은 믿는 것이 아니라 점검표로 대조하는 것이다.

8.3 point-in-time correctness

학습 목표

미래 누출이 무엇이고 왜 치명적인지 이해한다.

피쳐 타임스탬프의 의미(“알려진 시점”)와 시점 조인의 동작을 실측으로 확인한다.

직관: 그 시점에 알 수 있었던 것만

민원 급증 예측 모델의 훈련 데이터를 만든다고 가정한다. 훈련 샘플 하나는 “7월 2일 오전 9시의 강남구” 같은 (키, 시점) 쌍이고, 여기에 “전일 민원 건수” 피쳐를 붙여야 한다. 이때 7월 2일 하루 전체의 집계를 붙이면 어떻게 되는가 — 그 값은 7월 2일 오전 9시에는 **아직 존재하지 않았다**

(그날이 끝나야 확정된다). 미래의 정보가 과거 시점의 샘플에 포함된 것, 이것이 **미래 누출(future leakage)**이다. 누출된 모델은 훈련·평가에서는 비현실적으로 높게 나타나다가, 실제 서빙(미래를 알 수 없는 상황)에서 성능이 급격히 저하된다. 성능이 좋아 보여서 더 위험한 결함이다.

point-in-time correctness는 이것을 조회 규칙으로 막는다: 각 샘플의 기준 시점마다 **그 시점 이전 ($\leq$)에 알려진 가장 최근 피쳐 값만** 조인한다. 그래서 피쳐의 타임스탬프가 결정적으로 중요해진다 — 그 타임스탬프는 “값이 가리키는 기간”이 아니라 **“값이 알려진 시점”** 이어야 한다. 실습의 설계 결정이 정확히 이것이다: 7/1의 일별 집계는 7/1이 끝나야 확정되므로, 타임스탬프를 7/1이 아니라 **7/2 00:00**으로 부여했다. 2장의 이벤트 시간과 처리 시간, 6장의 watermark, 7장의 logical date에 이어 네 번째로 확인하는 시간의 구분이다 — 이번에는 “발생한 시각”도 “처리한 시각”도 아닌 **“알 수 있게 된 시각”**이다.

이것은 이 실습만의 규칙이 아니라 피쳐 스토어가 공통으로 내세우는 정확성 보장이다. Tecton은 이를 “각 훈련 샘플에 대해 그 시점의 세계 상태를 point-in-time correct하게 본다(point-in-time correct views of the state of the world)”고 표현하고, Hopsworks도 point-in-time correct join(시점 정합 조인)을 미래 누출(future data leakage)을 막는 절차로 설명한다(방법론 차용). 표현은 복잡하지만 뜻은 앞서 정의한 그대로다 — 각 샘플의 기준 시점 이전에 알려진 값만 붙인다. 실습이 하는 일은 이 보장을 최소 구성에서 직접 확인하는 것이다.

워크드 예제: 기준 시점 4개, 실측 조인

실습은 기준 시점이 다른 훈련 샘플 4개를 만들어 시점 조인을 실행했다 (ch7_feature_report.json). 수작업으로 먼저 추론한다 — 각 시점에 “알려진” 집계는 어디까지인가.

표 8.3 point-in-time 조인 관찰(실제 실행, 피쳐 타임스탬프 = 집계일 다음 날 00:00)

훈련 샘플(키, 기준 시점)	그 시점까지 확정된 집계	조인된 건수(실측)	검산(7장 실측)
관악구, 7/1 09:00	없음(첫 확정 = 7/2 00:00)	행 자체가 탈락	—
강남구, 7/2 09:00	7/1까지	8 (미매핑율 0.05)	7/1 강남 8
마포구, 7/3 09:00	7/2까지	5 (0.059)	7/2 마포 5
강남구, 7/4 09:00	7/3까지	9 (0.045)	7/3 강남 9

둘째 행 하나를 조인 규칙 그대로 수작업으로 따라간다. 샘플은 (강남구, 7/2 09:00)이다. ① 강남구의 피쳐 값들은 타임스탬프 7/2 00:00(=7/1 집계), 7/3 00:00, 7/4 00:00에 존재한다. ② 기준 시점 이전 ($\leq$ 7/2 09:00)인 것은 7/2 00:00 하나뿐이다 — 7/3·7/4의 값은 미래이므로 후보에서 제외된다. ③ 남은 후보 중 가장 최근인 7/2 00:00의 값 8이 붙는다. ④ TTL 확인: 기준 시점과의 간격 9시간 < 3일, 유효하다. 엔진의 실측 출력(8)이 이 손 검산과 정확히 일치한다 — 규칙을 이해했다면 표의 모든 행을 같은 방식으로 재현할 수 있다.

두 가지를 더 확인한다. 첫째, 강남구 샘플 두 개(7/2, 7/4)에 **다른 값**(8과 9)이 붙었다 — 같은 키라도 기준 시점이 다르면 “그때 알 수 있었던 값”이 다르기 때문이며, 이것이 시점 조인의 정의다. 7/2 09:00 샘플에 그날의 완성될 집계(7/2 강남 6)가 아니라 7/1의 8이 붙은 것이 바로 미래 누출 방지다. 둘째, 조인된 미매핑율이 날짜마다 다르다(0.05/0.059/0.045) — 품질 지표조차 “그 시점에 알려진 값”으로 붙는다.

관찰: 사라진 한 행 — 조인 후에는 행 수를 센다

표 8.3의 첫 행이 이 실습의 예상과 다른 실측이다. 7/1 09:00의 관악구 샘플은 조인할 피처가 아예 없는 시점(첫 확정 전)인데, 결과에서 피처가 빈 값(NaN)으로 붙는 것이 아니라 **행 전체가 제외되었다** — 입력 4행, 결과 3행이다(증거 파일의 `entity_rows_requested=4`, `historical_dropped_rows`에 탈락 행 기록). 훈련 데이터를 구성하는 관점에서 이것은 조용한 샘플 손실이다: 어떤 오류도 없이 훈련 셋이 감소하고, 특히 “이력이 짧은 신규 키”(새로 생긴 지역구·신규 가입자)의 샘플이 체계적으로 제외된다 — 무작위 손실이 아니라 **편향된 손실**이다. 대응은 정제(7.2)의 총계 보존과 같은 구조다: **조인 전후의 행 수를 반드시 대조하고, 차이가 나면 어느 샘플이 왜 빠졌는지 확인한다.** 실습 코드가 탈락 행을 계산해 증거 파일에 남기는 것이 그 구현이다.

한 가지 단서를 붙인다: “행째 탈락”은 이 실습이 검증한 로컬(파일 기반) 오프라인 스토어 구현의 실측 동작이다. 다른 오프라인 스토어 구현(웨어하우스 기반 등)에서는 같은 상황이 피처가 NaN인 행으로 남을 수도 있다 — 구현에 따라 증상이 다르다는 것 자체가, 조인 결과를 구현의 기본 동작에 의존하지 말고 행 수 대조를 자기 코드로 확인해야 할 또 하나의 이유다.

TTL: 너무 오래된 값도 붙이지 않는다

시점 조인에는 하한도 있다. 실습의 feature view는 TTL(time-to-live) 3일을 선언했다 — 기준 시점보다 3일 넘게 오래된 피처 값은 “그 시점에 알려져 있었더라도” 붙이지 않는다. 이유는 최신성이다: 석 달 전의 “전일 민원 건수”는 알려진 값이긴 해도 “전일”이라는 피처의 의미를 잃는다. TTL은 watermark(6장)와 같은 구조로 이해하면 정확하다 — watermark가 “얼마나 늦은 데이터까지 받을 것인가”라면, TTL은 “얼마나 오래된 피처까지 유효로 볼 것인가”다. 둘 다 선언 하나로 시간의 경계를 정하고, 경계 밖을 제외한다.

규칙대로 추론을 연장한다(실행이 아니라 정의에서 따라 나오는 결과다). 실습 피처의 마지막 값은 7/4 00:00(=7/3 집계)이다. 기준 시점이 7/5 09:00인 샘플에는 이 값이 붙는다(간격 1일 9시간 < 3일). 그러나 기준 시점이 7/8 09:00라면 간격이 4일 9시간으로 TTL을 넘어, 그 샘플은 표 8.3의 첫 행과 같은 상태 — 조인할 유효 값 없음 — 이 된다. 즉 **피처 공급이 중단되면(7장 배치 실패가 며칠 이어지면) TTL이 지난 시점부터 훈련 샘플이 조용히 제외되기 시작한다.** 배치 실패 대응 체크리스트(표 7.4)의 영향 범위 단계에 “피처 소비자”가 들어가야 하는 이유가 여기 있다.

기준 시점을 어디로 잡을 것인가

시점 조인의 결과는 피처 쪽 타임스탬프만이 아니라 **샘플 쪽 기준 시점을 어디로 잡느냐**에도 달려 있다. 실습은 기준 시점을 각 날짜의 09:00으로 설정했다. 임의로 선택한 값이 아니라 “그 시각에 예측을 내린다”는 업무 시나리오에서 도출한 값이다 — 오전 9시에 그날의 인력 배치를 정한다면,

모델이 볼 수 있는 것은 09:00까지 알려진 값뿐이다.

기준 시점을 잘못 설정하는 방식이 둘 있다. 하나는 **실제 예측 시각보다 늦게 설정하는 것이다**. 결정은 09:00에 내리는데 훈련 샘플의 기준 시점을 23:59로 두면, 그날 낮에 확정된 값이 훈련 샘플에 붙는다 — 조인 규칙은 그대로인데 설정값 하나로 미래 누출이 들어온다. 다른 하나는 **필요 이상으로 이르게 설정하는 것이다**. 자정 기준으로 훈련해 두고 서빙은 09:00에 하면 누출은 없지만, 서빙 쪽이 훈련보다 최신 값을 받는다 — 훈련-서빙 스큐의 시간 버전이다.

규칙은 하나로 정리된다: **훈련 샘플의 기준 시점은 운영에서 실제로 예측이 호출되는 시각과 같아야 한다**. 호출이 하루 한 번이면 그 시각 하나로, 사건마다 호출된다면 그 발생 시각 분포를 따라 표본을 구성한다. 09:00 한 시점으로만 훈련한 모델을 종일 호출하면, 아침 호출과 저녁 호출이 서로 다른 최신성 수준의 입력을 받는다.

훈련 데이터셋 생성의 최소 점검 절차

시점 조인으로 훈련 셋을 만들 때마다 통과해야 할 점검을 이 절의 실측에서 도출한다 — 이 네 줄이 훈련 데이터 검수의 최소선이다.

- ☐ 피쳐 타임스탬프의 의미가 "알려진 시점"으로 정의서에 명시되어 있는가(8.3의 전제)
- ☐ 조인 전후 행 수를 대조했는가 — 차이가 있으면 탈락 샘플의 목록과 사유를 확인(실습: 4→3, 신규 키 편향)
- ☐ 같은 키의 샘플들이 기준 시점에 따라 다른 값을 받는지 표본 검사했는가(실습: 강남 8과 9 — 모두 같다면 시점 조인이 아니라 최신값 조인을 하고 있는 것)
- ☐ TTL 밖 구간(피쳐 공급 중단 기간)의 샘플 손실을 확인했는가

공공 사례

정책 효과 예측 모델을 사후 검증하는 감사 상황을 가정한다. "이 모델의 훈련 데이터에 발표 전 통계가 들어가지 않았는가"라는 질문에, 각자 계산 구조에서는 훈련 코드를 줄 단위로 재감사해야 답할 수 있다. 피쳐 스토어 구조에서는 두 가지로 답한다: 피쳐 타임스탬프의 의미가 "확정·공개 시점"으로 문서화되어 있고(피쳐 정의서, 8.4), 훈련 조회가 point-in-time 조인 API를 통해서만 일어났다는 것. 미래 누출 방지가 개인의 주의에서 **구조와 문서**로 옮겨 간 것이며, 이것이 검수 가능한 시스템의 조건이다. ### 핵심 정리 - 미래 누출은 훈련 시점에 알 수 없던 값이 피쳐로 붙는 것 — 훈련에서는 좋아 보이고 서빙에서 성능이 떨어진다. - 피쳐 타임스탬프는 "값이 알려진 시점"이어야 하고, 시점 조인은 기준 시점 이전의 최근 값만 붙인다(TTL은 최신성 하한). - 조인할 값이 없는 샘플은 조용히 탈락할 수 있다 — 조인 전후 행 수 대조는 필수 검수다.

8.4 공공 데이터 피쳐의 소유자와 접근 권한

학습 목표

피쳐 정의서에 담아야 할 거버넌스 항목을 안다.

부서 간 피처 재사용의 책임 경계를 이해한다.

핵심 개념

피처가 “한 곳에 정의되고 여러 모델이 재사용”하는 자산이 되는 순간, 기술 문제가 아니라 거버넌스 문제가 함께 온다. 재사용은 효율이지만, 그 피처가 틀렸을 때 영향도 함께 커진다 — 원천 부서가 집계 방식을 바꿨는데 소비하는 모델 세 개가 그것을 모른다면. 그래서 피처 정의서에는 계산 명세만이 아니라 **소유와 책임**이 들어가야 한다.

소유자(owner): 이 피처의 정의·품질에 책임지는 부서/담당. 원천이 바뀌면 소비자에게 알릴 의무가 여기에 붙는다.

원천(source)과 갱신 주기: 어느 시스템의 어떤 산출물에서, 얼마나 자주 — 실습 피처라면 “7장 일별 배치, 일 1회(마감 후)”.

개인정보 등급: 피처 단위로 개인정보 포함 여부를 표기한다. 지역 단위 집계(실습)는 비식별이지만, 가구·개인 단위 피처는 접근 통제와 이용 목적 제한이 붙는다(보충). 연구용 동의로 수집된 데이터의 피처를 상용·타 목적 모델이 조회하는 것은 기술적으로 가능해도 법적으로 불가할 수 있다 — 피처 스토어의 조회 권한이 이 경계를 강제하는 지점이다.

타임스탬프 의미: 8.3의 “알려진 시점” 규약도 정의서에 명시한다 — 미래 누출 감사의 근거 문서가 된다.

실습은 이 항목들을 feature view의 태그로 선언하고 registry에서 추출해 정의서로 내보냈다 (ch7_feature_definitions.json). 그 내용이 표 8.4다 — 소유자 표기는 시뮬레이션(가상 부서)임을 명시한다. 이 JSON은 이 장의 종착점이 아니라 13장의 입력이다: 13장의 피처 카탈로그는 여기서 나온 소유자·원천·개인정보 등급을 받아 접근 통제와 감사 로그로 집행한다 — 정의(8장)와 집행(13장)이 같은 명세를 공유하면 거버넌스 문서와 시스템이 분리되지 않는다.

표 8.4 피처 정의서(실습 등록 내용, registry에서 추출)

항목	값
프로젝트 / 서비스	public_complaints / complaint_model_v1
entity(조인 키)	district(lawd_cd) — 법정동 시군구 코드(7.2의 표준 키)
피쳐 1	complaint_count(Int64) — 전일 확정 민원 건수(7장 일별 집계)
피쳐 2	day_unmapped_rate(Float64) — 전일 정제 품질·매핑 실패율(7장 quality_check)
타임스탬프 의미	값이 확정·공개된 시점(집계 대상일 다음 날 00:00 UTC)
TTL	3일(최신성 하한)
소유자 / 원천 / 주기	민원데이터팀(가상) / 7장 일별 배치 확정 산출물 / 일 1회
개인정보	없음(지역 단위 집계)

조인 키는 지역명이 아니라 법정동코드다. 7.2에서 “이름이 아니라 코드로 붙인다”고 했던 표준 코드 원칙이, 피쳐 스토어의 entity 설계에서 그대로 반복된다. 기관 간·시스템 간 피쳐 결합이 가능한 것은 이 키 규율 덕분이다.

민간에서는 같은 문서를 **피쳐 카탈로그(feature catalog)**라 부르고, ML 플랫폼의 검색·소유자 관리 기능으로 구현한다. 담기는 항목은 표 8.4와 거의 같다. 다른 것은 그 문서를 두는 이유다 — 민간에서는 팀이 늘 때 같은 피쳐를 각자 다시 만드는 낭비를 줄이는 효율 도구이고, 공공에서는 감사·정보공개 요구에 답하는 근거 문서다. 그래서 공공 정의서에는 소유 부서와 타임스탬프 의미가 빠질 수 없고, 민간 카탈로그에서는 검색 태그와 사용 중인 모델 목록이 더 중요해진다.

접근 권한은 피쳐의 개인정보 등급에 따라 계층화한다. 아래는 공공 기관에서 쓸 수 있는 등급 설계의 예시다(실습 피쳐는 전부 1등급 — 값은 예시 설계이지 실측이 아니다).

표 8.5 피쳐 접근 등급 설계(예시)

등급	피쳐 성격	예	조회 주체	통제 장치
1. 공개 집계	지역·기관 단위 통계	지역구별 일별 민원 건수	전 부서 모델	정의서 공개, 로그만
2. 내부 업무	업무 민감(비개인)	처리 지연 중인 건수 내역	소관 부서 승인 모델	조회 권한 + 소유자 승인
3. 개인정보	개인·가구 단위	가구별 체납 이력	법적 근거 있는 모델만	목적 제한, 접근 로그 감사, 보존 기한

핵심은 등급이 피처 단위로 부여된다는 것이다 — 시스템 단위 권한(“피처 스토어에 접근 가능”)은 3등급 피처의 존재 앞에서 무의미하다. 정의서의 privacy 항목(표 8.4)이 이 등급의 선언 위치다.

entity 키를 무엇으로 잡을 것인가

entity는 형식상 조인 키의 선언이지만, 실제로 선택하는 것은 **피처가 서술하는 대상의 단위**다. 실습은 지역구를 선택했다. 같은 민원 데이터로 다른 단위도 선택할 수 있다 — 민원 유형별, 처리 부서별, 개별 민원 건별. 무엇이 옳은지는 데이터가 아니라 **서빙 시점에 무엇을 키로 조회하게 되는가**가 정한다. 상황실이 “강남구의 현재 상태”를 묻는다면 키는 지역구여야 하고, 접수 창구가 “이 민원 건의 예상 처리 시간”을 묻는다면 키는 민원 건 ID여야 한다. 서빙 시점에 존재하지 않는 값을 키로 설정하면 그 피처는 조회 자체가 불가능하다.

단위를 세분화할수록 피처는 정밀해지지만 대가가 둘 따른다. 온라인 스토어의 키 개수가 늘어 적재 시간과 저장 비용이 커지고, 키마다 이력이 짧아져 시점 조인에서 탈락하는 샘플이 늘어난다(8.3의 행 탈락). 반대로 굵게 설정하면 조회는 가볍지만 같은 지역구 안의 차이를 모델이 반영하지 못한다. 공공에는 조건이 하나 더 붙는다 — 개인·가구 단위 키는 그 자체로 3등급 피처(표 8.5)가 되어, 정밀도를 얻는 대신 접근 통제와 이용 목적 제한을 함께 적용받는다. 4장의 파티션 키 선택과 같은 구조다: 세분화에 비용이 따른다는 것, 그리고 무엇을 한 단위로 볼 것인가가 기술이 아니라 업무 판단이라는 것.

feature service: 무엇을 한 묶음으로 낼 것인가

feature view가 피처의 정의라면, feature service는 **한 모델이 소비하는 묶음**이다. 실습은 view 하나·피처 둘을 `complaint_model_v1` 하나에 묶었으므로 선택의 여지가 없었다. 피처가 수십 개로 늘고 모델이 여럿이 되면 묶는 기준을 정해야 한다.

두 방식에는 다음과 같은 차이가 있다. **모델 단위로 묶으면** — 모델 하나가 쓰는 피처 전부를 한 service로 — 서빙 코드가 호출 한 번으로 필요한 것을 모두 받고, 정의를 바꿨을 때 영향 범위가 그 모델 하나로 좁혀진다. 대신 여러 모델이 공유하는 피처가 service마다 중복 선언된다. **주제 단위로 묶으면** — 민원 피처 묶음, 기상 피처 묶음 — 재사용이 쉽지만 모델이 여러 service를 조합해 호출해야 하고, 어느 모델이 무엇을 쓰는지 추적이 흐려진다.

판단의 기준값은 **피처 재사용률**이다 — 전체 피처 중 두 개 이상의 모델이 공유하는 피처의 비율. 이 비율이 낮으면 모델 단위가 관리하기 쉽고, 상승하면 주제 단위로 묶고 모델이 조합해 사용하는 편이 중복 선언을 줄인다. 다만 재사용률이 높다는 것은 정의 변경 하나가 영향을 미치는 모델 수도 많다는 뜻이다 — 효율과 영향 범위가 같은 방향으로 커지므로, 재사용률이 올라갈수록 아래의 버전 규율이 더 필요해진다.

정의가 바뀔 때: 버전으로 말한다

피처 정의는 불변이 아니다 — 정제 규칙이 바뀌면(예: 7.2의 별칭 사전 확장) 같은 이름의 피처가 다른 값을 갖게 된다. 이때 기존 정의를 조용히 덮어쓰면, 그 피처로 훈련된 기존 모델들의 훈련-서빙 일치가 소급으로 무효화된다는 — 스큐를 막으려던 구조가 스큐의 원인이 되는 역설이다. 규율은 버전이다: 정의 변경은 새 feature service 버전(`complaint_model_v2`)으로 내고, 기존 v1은 소비

모델들이 이전을 마칠 때까지 유지한다. 실습의 service 이름에 v1이 붙어 있는 것은 장식이 아니라 이 규율의 자리 표시이며, “무엇이 언제 왜 바뀌었나”의 기록 체계는 9장(레지스트리)에서 본격화된다.

공공 사례

A부서가 만든 “지역별 민원 피처”를 B부서의 재난 대응 모델이 재사용한다고 가정한다. 정의서가 없으면 B부서는 피처의 정제 규칙(미매핑 분리? 중복 제거?)을 모른 채 소비하고, A부서는 자신의 집계 변경이 B의 모델을 흔든다는 것을 모른다. 정의서가 있으면 계약이 성립한다: A는 정의 변경 시 버전을 올리고(complaint_model_v1의 v1이 그 자리다), B는 명세를 읽고 적합성을 판단한다. 발주·검수 문서에서 “피처를 재사용한다”는 문장을 확인하면 물어야 한다 — 정의서는 어디 있고, 소유자는 누구이며, 변경은 어떻게 통보되는가. ### 역방향: 이 장의 설계를 민간 현장에 옮기면 지금까지는 민간 방법을 공공 문제에 적용했다(8.2c). 반대 방향도 성립한다 — 이 장에서 만든 설계를 그대로 민간 현장에 적용하면 무엇이 바뀌고 무엇이 유지되는지 세 현장에서 확인한다.

추천 시스템의 이용자 행동 피처는 entity가 지역구에서 이용자 ID로 바뀔 뿐 구조가 같다. 이용자가 최근 10분간 본 상품 카테고리를 집계해 온라인 스토어에 적재하고, 추천 API가 밀리초 안에 읽어 모델에 입력한다. 훈련은 같은 정의를 과거 이력에 시점 조인해 만든다 — 두 경로가 같은 정의를 참조한다는 8.1의 요구가 그대로다. 바뀌는 것은 두 가지다. entity를 개인 단위로 잡았으므로 실습의 1등급 피처가 여기서는 개인정보 등급 피처가 되고, materialize 주기는 하루가 아니라 분 단위로 내려간다. 이용자 행동은 하루면 낡기 때문이다.

카드 사기 탐지 피처는 TTL과 최신성 요구 수준이 가장 높은 경우다. “같은 카드의 직전 5분 거래 횟수”, “직전 1시간 거래 금액 합” 같은 피처로 승인 여부를 수십 밀리초 안에 판정한다. 여기서 point-in-time 조인은 선택이 아니다 — 미래 누출이 섞이면 훈련·평가에서 정밀도가 비현실적으로 높게 나오고, 배포 후 성능이 급락한다. 8.3에서 관악구 샘플이 조용히 탈락한 것과 같은 현상도 그대로 재현된다. 이력이 짧은 신규 카드의 샘플이 체계적으로 빠지고, 그러면 모델은 신규 카드 사기를 배우지 못한 채로 배포된다. 편향된 손실이라는 진단이 도메인과 무관하게 같다.

실시간 개인화(콘텐츠·가격·프로모션)는 배치 피처와 실시간 피처를 한 모델이 함께 입력으로 받는 경우다. 과거 구매 이력은 7장식 일별 배치에서, 현재 세션 행동은 6장식 스트리밍 집계에서 온다. 두 계통의 피처를 하나의 feature service로 묶어야 서빙이 호출 한 번으로 둘 다 읽는다 — 앞 소절의 묶기 판단이 실제로 쓰이는 자리다. 최신성 요구가 다른 두 view를 한 service에 묶으므로, service의 최신성은 둘 중 느린 쪽이 정한다는 점을 설계에 적어 두어야 한다.

이 셋을 관통하는 것은 하나다. **entity 키의 단위, 피처 타임스탬프의 의미, TTL과 materialize 주기, feature service의 묶음** — 이 네 결정은 어느 현장에서도 똑같이 내려야 하고, **근거만 도메인에 따라 달라진다**. 공공에서 그 근거가 감사 재현 가능성·형평성·법적 이용 목적이라면, 민간에서는 매출·응답 SLA·인프라 비용이다. 방향의 비대칭도 4장과 같다 — 민간 설계를 공공에 도입하면 증거 산출 의무가 한 겹 추가되지만, 공공 설계를 민간에 도입하면 그 의무를 제외해도 손실이 없다. 피처 정의서는 피처 카탈로그가 되고, 일관성 점검표는 스쿼 알림이 되며, 시점 조인 규율은 그대로 모델 성능 과대 추정을 막는다.

핵심 정리

피처 재사용은 효율이자 위험 — 정의서(소유자·원천·주기·개인정보·타임스탬프 의미)가 계약이다.

entity 조인 키는 표준 코드(7.2)로 — 이름 기반 키는 피처 결합에서도 같은 이유로 실패한다.

entity 단위·TTL·materialize 주기·service 묶음은 도구 설정이 아니라 서빙 질의·배치 실패 허용·재사용률에서 도출하는 설계 결정이다.

조회 권한은 성능 장치가 아니라 이용 목적 제한을 강제하는 거버넌스 장치다.

실습 8.1 Feast 기반 최소 피처 저장소 설계와 실행(★★★☆☆)

이 실습은 **설계 → 실행 → 대조 → 위험 식별** 네 단계로 진행한다. 순서에 의도가 있다. 이미 정해진 entity와 TTL로 스크립트부터 실행하면 그 값이 왜 그 값인지 묻지 않게 되고, 조회 결과는 “도구가 산출한 근거 없는 수치”로 남는다. 먼저 자기 판단으로 설계하고 조회 결과를 예측한 다음 실행 산출물과 대조하면, 어긋난 지점이 곧 자기 설계의 결함이 된다. 감사·검수는 그 뒤 4단계에서 다룬다 — 설계 근거와 실행 증거가 모두 확보되어야 검수 질문에 답할 수 있기 때문이다.

실습 목표

요구사항에서 entity 키·TTL·집계 창·materialize 주기·feature service 묶음을 **스스로 정하고 근거를 적는다**(설계).

7장 확정 집계를 원천으로 entity-feature view·feature service를 등록하고, point-in-time 조인·materialize·온라인 조회를 실행한다(실행).

설계 단계에서 예측한 조인 결과 행 수·온라인 행 수·조인 값을 실행 산출물과 대조하고, 어긋난 원인을 설명한다(대조).

훈련·서빙 일관성 점검표와 위험 식별 표로 설계 근거와 실행 증거를 검수 언어로 옮긴다(증거).

입력 데이터

피처 원천은 7장 실습의 실측 산출물(일별 집계·품질 기록 3일치) 스냅샷이며

`data/input/ch6_daily/`에 동봉했다 — 민원 이벤트 자체가 4~7장과 동일한 시뮬레이션 입력이라는 표기도 그대로 승계된다. 피처 정의서의 소유자 표기도 가상이다. 이 실습의 관찰 대상은 실제 Feast 엔진(0.64.0)의 등록·시점 조인·적재·조회 동작이며, 인용 수치는 모두 실제 실행 산출물에서 산출된다.

1단계 설계 — 피처 설계표와 조회 결과 예측

스크립트를 실행하기 전에 설계를 먼저 확정한다. 아래 요구사항을 읽고 **표 8.6의 빈칸을 직접 채운다**. 표 8.4는 실습이 등록한 완성 예시이므로 그대로 옮겨 적지 않는다 — 채운 뒤에 비교한다.

요구사항(가상 발주 조건) - 상황실 대시보드에서 지역구 하나를 선택하면 그 구의 최신 피처를 제시한다. 응답은 1초 안이어야 한다. - 민원 급증 예측 모델은 매일 오전 9시에 그날의 인력 배치를 산출한다. 재훈련은 분기마다 지난 이력 전체로 한다. - 피처 원천은 7장 일별 배치의 확정 산출물이고, 배치는 매일 새벽에 전일분을 마감한다. - 배치가 하루 실패한 적은 있고, 이를 연속 실패한 적은 없다. 복구는 다음 날 이뤄진다. - 감사 요구: 과거 어느 예측이든 “그 시점에 알 수 있었던 값으로 계산되었는가”를 사후에 확인할 수 있어야 한다.

표 8.6 피처 설계표 — 학습자 작성용

항목	내가 정한 값	근거(요구사항의 어느 문장에서 나왔는가)
entity(조인 키)와 그 단위		
피처의 집계 창(값이 가리키는 기간)		
피처 타임스탬프의 의미		
feature view의 TTL		
materialize 주기		
feature service에 묶을 피처		
훈련 샘플의 기준 시점		
이 설계가 깨지는 조건		

작성할 때 지킬 것이 셋이다. 첫째, 근거 칸에 “기본값이라서” 또는 “보통 그렇게 한다”를 쓰지 않는다 — 요구사항의 문장 하나를 제시해야 한다. 둘째, TTL은 디스크 사정이나 성능이 아니라 **배치 실패를 며칠까지 견딜 것인가**에서 역산한다(8.3). 셋째, “이 설계가 깨지는 조건”에는 자기 선택이 불리해지는 상황을 적는다 — 트레이드오프를 이해하고 선택했는지 확인하는 칸이다.

조회 결과 예측: 실행 스크립트는 3일 × 3개 지역구 = 9행의 피처 원천을 생성하고, 기준 시점이 서로 다른 훈련 샘플 4행으로 시점 조인을 한 뒤, materialize해서 온라인 조회를 한다. 실행하기 전에 아래 세 칸을 채운다.

항목	내 예측	실제(3단계에서 기입)
시점 조인 결과 행 수(입력 4행)		
materialize 후 온라인 스토어 행 수(원천 9행)		
강남구 두 샘플(7/2 09:00, 7/4 09:00)에 붙는 값		

각 칸에 예측 근거도 한 줄씩 적는다. 근거 없는 수치는 대조해도 학습 효과가 없다.

2단계 실행 — Feast 최소 구성 실행

Python 3.10 이상(이 실습은 3.13에서 검증)이면 되고, 외부 인프라는 필요 없다 — offline store는 parquet 파일, online store는 SQLite, registry는 로컬 파일이다.

```
cd practice/chapter7
python3 -m venv venv && source venv/bin/activate
pip install -r code/requirements.txt
python run_chapter7.py
```

스크립트는 다음 순서로 진행하며, 각 단계가 절 하나씩에 대응한다.

단계	내용	대응 절
1	7장 집계 → 피쳐 원천 parquet(9행), 타임스탬프 = 다음 날 00:00	8.3(알려진 시점)
2	entity-feature view-service 정의, apply로 registry 등록	8.1(정의 일원화)
3	기준 시점 4개로 get_historical_features — 시점 조인·행 탈락 관찰	8.3
4	materialize(범위 명시) → 온라인 조회 → 오프라인 최신값과 대조	8.2
5	피쳐 정의서·일관성 점검표·증거 파일 출력	8.4

핵심 코드

피쳐 정의에 필요한 네 항목이다 — “무엇을(스키마), 어디서(source), 언제 기준으로(timestamp), 얼마나 최신으로(ttl)”가 선언 하나에 담긴다. 1단계 표 8.6에서 선택한 값이 들어가는 자리가 여기다.

```
FeatureView(name="complaint_daily", entities=[district], ttl=timedelta(days=3),
            schema=[Field(name="complaint_count", dtype=Int64),
                    Field(name="day_unmapped_rate", dtype=Float64)],
            source=FileSource(path=str(parquet), timestamp_field="event_timestamp"))
```

전체 코드는 [practice/chapter7/code/7-1-feature-store-minimal.py](#) 참고

실행 중 확인 체크리스트

- ☐ 실행 전에 표 8.6과 조회 결과 예측을 적어 두었는가(실행 후에 적으면 대조가 성립하지 않는다)
- ☐ 피쳐 원천이 9행(3일×3지역)으로 생성되었는가
- ☐ feature_repo/ 에 registry.db(정의)와 online_store.db(값)가 따로 생성되었는가 — 정의·값 분리의 실물
- ☐ apply 후 entity-view-service가 등록 로그에 기록되었는가

- 시점 조인: 입력 4행 → 결과 3행, 탈락 1행이 (관악구, 7/1 09:00)인가 — 탈락은 `ch7_feature_report.json` 의 `historical_dropped_rows`에서만 확인된다
- 강남구 두 샘플에 다른 값(8, 9)이 붙었는가 — 미래 누출 방지의 직접 증거
- 온라인 조회 3건이 오프라인 최신값과 모두 일치(`all_consistent=true`)하고 마지막 줄이 `CH8_RUN_PASS` 인가

산출물 안내

산출물	내용	용도
<code>complaint_daily_features.parquet</code>	피처 원천(offline store) 9행	시점 조인의 근거 데이터
<code>feature_repo/</code>	registry-online SQLite·설정	정의 등록과 온라인 적재의 실체
<code>ch7_feature_definitions.json</code>	피처 정의서 (registry에서 추출)	8.4 거버넌스 자산 — 표 8.4의 원본
<code>ch7_consistency_report.json</code>	훈련-서빙 일관성 점검표	표 8.2b의 원본
<code>ch7_feature_report.json</code>	시점 조인 결과·탈락 행·버전	이 장 인용 수치의 증거 파일
표 8.6 피처 설계표(학습자 작성)	entity·집계 창·TTL·주기·묶음과 각각의 근거	피처 설계표 — 검수 자산, 3단계 대조 후 확정

실행 결과(실제)

시점 조인(입력 4행): 강남@7/2 09:00 → **8**(0.05), 마포@7/3 09:00 → **5**(0.059), 강남@7/4 09:00 → **9**(0.045). 관악@7/1 09:00은 결과에서 **행째 탈락**(`historical_dropped_rows`에 기록) — 표 8.3.

`materialize(2026-07-01~현재 범위 명시)` 후 온라인 조회: 마포 6, 관악 5, 강남 9(모두 미매핑율 0.045) — 오프라인 최신값(7/3 집계)과 3키 모두 일치, `all_consistent=true` — 표 8.2b.

등록 정의: `project=public_complaints`, `entity=district(lawd_cd)`, `view=complaint_daily`(TTL 3일, 피처 2개), `service=complaint_model_v1` — 표 8.4.

실행 시간은 수 초 수준이다(Spark 기동이 있던 6장과 대조적) — 피처 스토어의 무게는 계산이 아니라 규율에 있다는 8.1의 서술과 부합한다. 피처 원천 `parquet`은 9행이며, 시점 조인·온라인 조회의 모든 값이 그 9행 안에서 설명된다.

관찰

조인된 값이 모두 7장 실측에서 수작업으로 검산된다: 8=7/1 강남, 5=7/2 마포, 9=7/3 강남, 6=7/3 마포. 피처 스토어는 값을 생성하지 않았다 — 확정 산출물을 시점 규율에 맞춰 전달했을 뿐이다. 이 “계산하지 않음”이 훈련-서빙 일치의 원천이다.

registry(정의)와 store(값)의 분리를 실물로 확인한다: `feature_repo/registry.db` 가 정의, `parquet-SQLite`가 값이다. 정의만 바뀌어도, 값만 바뀌어도 각각 사고가 된다 — 9장의 버전 관리 사고방식이 필요한 이유다.

피처 정의서(`ch7_feature_definitions.json`)는 수작업으로 쓴 문서가 아니라 registry에서 추출한 것이다 — 문서가 실물과 어긋나는 고전적 문제(문서 따로, 코드 따로)를 “문서를 실물에서 생성”으로 막는 패턴이며, 7장의 증거 파일과 같은 철학이다.

실행 환경에서 발생한 오류 사례

이 실습을 준비하며 실제로 발생한 오류의 경과를 기록해 둔다. 8.2에서 제시한 빈 온라인 스토어는 처음에 `materialize` 단계에서 드러나지 않았다 — 적재 명령은 정상 종료 로그를 남겼다. 문제는 그다음, 온라인 조회값을 정수로 캐스팅(casting)하는 줄에서 `int() argument must be ... not 'NoneType'` 예외가 발생했다. 즉 **오류가 난 곳과 원인이 있는 곳이 달랐다**(원인: 적재 창 이탈, 발현: 조회값 사용 지점). 여기서 조회 코드가 “값이 없으면 0으로” 같은 방어적 기본값을 사용했다면 어떻게 되었을지 가정한다 — 예외조차 없이, 모든 지역구의 피처가 0인 채로 모델이 서빙되었을 것이다. 방어적 기본값은 편리하지만 침묵 실패를 한 층 더 깊이 숨긴다 — 피처 조회의 결측은 기본값으로 대체하지 말고 계측(결측률 지표)하고, 임계 초과 시 서빙을 막는 것이 안전한 쪽이다 (11장 모니터링과 연결).

3단계 대조 — 예측과 실측의 대조

1단계 예측표의 “실제” 열을 실행 산출물로 채운다. 조인 결과 행 수와 탈락 행은

`ch7_feature_report.json` 의

`entity_rows_requested · historical_join · historical_dropped_rows` 에, 온라인 조회는

`ch7_consistency_report.json` 의 `rows` 에 있다.

항목	실측
시점 조인 결과 행 수(입력 4행)	3 (탈락 1행: 관악구 7/1 09:00)
materialize 후 온라인 스토어 행 수(원천 9행)	3 (지역구당 최신 1행)
강남구 두 샘플에 붙는 값	8(7/2 09:00) / 9(7/4 09:00)

세 칸에서 각각 다른 내용을 확인할 수 있다.

조인 행 수를 4로 적었다면 무엇을 빠뜨렸는가. 시점 조인은 입력 행 수를 보존하지 않는다. 조인할 유효 피처가 없는 샘플은 값이 빈 채로 남는 것이 아니라 결과에서 사라진다(8.3). 훈련 셋을 구성하는 쪽에서는 조용한 샘플 손실이고, 손실이 무작위가 아니라 이력이 짧은 키에 집중된다는 점이 문제다. 이 실습에서 사라진 것은 첫 확정 이전 시점의 관악구 샘플 하나였다. 앞의 단서 그대로, 이 “행째 탈락”은 로컬 파일 기반 오프라인 스토어의 실측 동작이다 — 구현에 따라 NaN 행으로 남을 수도 있으므로, 행 수 대조를 자기 코드로 포함하는 것이 대응이다.

온라인 행 수를 9로 적었다면 무엇을 빠뜨렸는가. materialize는 이력을 복사하는 작업이 아니라 **키별 최신값만 남기는 작업**이다. 원천 9행이 3행이 되는 것이 정상이고, 9행이 나왔다면 오히려 설정이 잘못된 것이다. 온라인 스토어는 오프라인 스토어의 작은 사본이 아니라 성격이 다른 저장소라는 8.2의 서술이 여기서 수치로 확인된다. 표 8.6에 materialize 주기를 적었다면, 그 주기가 곧 이 3행이 낡는 속도다.

강남구 두 값을 같게 적었다면 무엇을 빠뜨렸는가. 같은 키라도 기준 시점이 다르면 “그때 알 수 있었던 값”이 다르다. 두 값이 같게 나온다면 시점 조인이 아니라 최신값 조인을 하고 있는 것이고, 그것이 곧 미래 누출이다.

TTL 선택도 여기서 검산한다. 요구사항은 “배치가 하루 실패한 적은 있고, 이를 연속은 없다”였다. 표 8.6에 TTL을 1일로 적었다면, 배치가 하루 실패한 다음 날 기준 시점부터 조인할 유효 값이 사라져 훈련 샘플 전체가 제외된다 — 8.3의 TTL 추론이 그대로 적용된다. 실습이 3일을 사용한 것은 이를 연속 실패까지 견디고 하루의 여유를 남기기 위해서다. TTL을 길게 설정하면 안전하지만, 대신 오래된 값이 “전일 건수”라는 이름으로 사용되는 최신성 저하를 감수한다.

대조가 끝나면 표 8.6으로 돌아가 **“이 설계가 깨지는 조건” 칸을 다시 쓴다.** 처음에 비워 두었거나 막연히 적었다면, 지금은 행 탈락·온라인 3행·TTL 검산을 근거로 작성할 수 있다. 제출하는 설계표는 이 수정을 반영한 최종본이다.

4단계 위험 식별 — 설계와 증거를 검수 언어로 옮기기

앞 세 단계에서 설계 근거(1단계)와 실행 증거(2·3단계)를 모두 확보했다. 검수·검사가 요구하는 것은 그 둘을 이어 붙인 형태다 — “무엇이 잘못될 수 있는가, 그렇게 판단한 근거가 어느 산출물의 어느 값인가, 무엇으로 막는가”. 앞 네 행은 실행 증거에서, 마지막 행은 1단계 설계에서 나온다.

먼저 이 실습이 운영 대비 축소된 지점을 명시해 둔다. 저장소 구현(parquet·SQLite ↔ 웨어하우스·Redis), 적재 스케줄(수동 1회 ↔ 7장식 DAG 정기 실행), 접근 통제(없음 ↔ 표 8.5의 등급별 권한), 피처 규모(2개 ↔ 수백 개·다중 view). 축소된 것은 규모와 인프라이지 규율이 아니다 — 시점 조인·일관성 대조·정의서는 어느 규모에서든 같은 형태로 적용된다.

위험	실습에서의 근거	대응 방향
훈련-서빙 피쳐 불일치	점검표로 3키 일치 확인(불일치면 검출)	일관성 대조를 배치에 상시 편입 (11장 감시)
미래 누출	시점 조인이 7/2 샘플에 7/1 값만 부여	타임스탬프="알려진 시점" 규약 + 정의서 명시, 기준 시점을 예측 호출 시각에 맞춤
조용한 샘플 손실	시점 조인 4행→3행, 탈락은 증거 파일로만 확인	조인 전후 행 수 대조 의무화
빈 온라인 적재	materialize 창 이탈 시 조회가 None(실제 경험)	적재 건수 확인 + 서빙 전 일관성 점검, 결측률 지표화
TTL-집계 창 과소 설정으로 인한 훈련 샘플 소실	1단계 설계 + 3단계 TTL 계산: TTL이 배치 실패 허용 일수보다 짧으면 실패 다음 날부터 유효 값이 사라짐	TTL을 최대 연속 배치 실패 허용 일수보다 길게, 피쳐 공급 중단을 감시 대상에 포함(11장)

셋째 열이 비어 있거나 “모니터링한다” 같은 말로 채워지면 그 행은 검수 문서에서 유용하지 않다. 대응 방향에는 **바꿀 설정값이나 바꿀 설계 결정**이 들어가야 한다.

제출물 3종: ① 표 8.6 피쳐 설계표(3단계 수정 반영한 최종본), ② `ch7_feature_report.json` 과 `ch7_consistency_report.json`, ③ 위 위험 식별 표. ①이 설계, ②이 증거, ③가 둘을 이은 검수 자산이다.

보충(전문가 트랙 포인터)

훈련-서빙 스큐 검증: 표 8.2b는 저장 값의 일치를 확인한다. 운영에서는 서빙 시점에 실제로 모델에 들어간 피쳐 값을 로깅해 훈련 분포와 정기 비교한다(스큐의 조기 발견) — 11장 드리프트 감시와 같은 도구를 쓴다. 분포 거리는 PSI(Population Stability Index, 모집단 안정성 지수) 같은 지표로 측정하고, 임계를 넘으면 피쳐 계산이 어긋났거나 원천 데이터가 변한 신호로 해석한다.

피쳐 lineage와 메타데이터 관리: 피쳐 정의서의 확장판 — 원천 테이블부터 피쳐까지의 변환 계보(lineage)를 기록하면 “이 피쳐가 틀렸을 때 어디를 고치고 누구에게 알리는가”가 그래프로 답해진다. 7장 룰업의 계층화(하위 확정 산출물만 읽기)가 lineage의 한 단면이다.

개인정보 포함 피쳐의 제한적 사용 정책: 가구·개인 단위 피쳐는 수집 목적 내 이용, 접근 권한 분리, 보존 기한이 붙는다. 피쳐 스토어의 조회 권한·감사 로그가 정책의 기술적 집행 지점이며, 연구용(IRB 동의) 데이터와 상용 모델의 경계는 13장에서 확장한다.

핵심 용어

피쳐(feature): 모델 입력으로 가공된 값 — 원천 데이터와 모델 사이의 계약

훈련-서빙 스큐: 훈련과 서빙의 피쳐 계산이 어긋나 생기는 성능 저하
entity / feature view: 조인 키의 선언 / 원천·스키마·타임스탬프·TTL을 묶은 피쳐 정의
offline / online store: 과거 이력 전체(훈련용) / 키별 최신값(서빙용)
materialize: 오프라인 최신값의 온라인 적재 — 범위·건수 확인 필수
point-in-time 조인: 기준 시점 이전에 알려진 최근 값만 붙이는 훈련용 조회
미래 누출: 그 시점에 알 수 없던 값이 훈련 피쳐에 포함되는 것 — 훈련에서만 성능이 높게 나타난다
일관성 점검표: 오프라인 최신값과 온라인 조회값의 키별 대조 기록 — 이중 저장의 품질 게이트
TTL: 시점 조인의 최신성 하한 — 이보다 오래된 값은 붙이지 않는다
훈련-서빙 스큐와 미래 누출: 전자는 훈련·서빙의 계산 불일치, 후자는 시간 규율 위반 — 해법도 각각 일원화(8.1)와 시점 조인(8.3)으로 다르다
registry: 피쳐 정의(코드)의 등록소 — 값이 아니라 정의의 버전·감사 지점
feature service: 모델이 소비하는 피쳐 묶음의 버전 단위(실습: complaint_model_v1)
피쳐 정의서: 스키마+소유자+원천+주기+개인정보+타임스탬프 의미의 계약 문서
피쳐 최신성 / 결측률 / 재사용률: 온라인 값이 얼마나 낡았는가 / 조회가 값을 찾지 못한 비율 / 두 모델 이상이 공유하는 피쳐의 비율 — 앞의 둘은 운영 감시 지표, 마지막은 service 묶기 판단의 기준값

검수 질문 한 줄 요약

발주·검수 문서에서 이 장의 내용은 세 질문으로 압축된다: ① 훈련과 서빙이 같은 피쳐 정의를 참조하는가(구현이 둘이면 스큐), ② 피쳐 타임스탬프는 “알려진 시점”인가(아니면 미래 누출), ③ 오프라인·온라인 값의 일치를 무엇이 정기적으로 대조하는가(표 8.2b의 자동화). 셋 다 답이 없으면 “피쳐 스토어를 쓴다”는 문장은 도구 이름일 뿐 구조가 아니다.

다음 장 예고

이 장의 피쳐 정의에는 이미 버전이 붙어 있었다 — complaint_model_v1. 피쳐가 바뀌면 모델을 다시 훈련해야 하고, 그러면 “어느 데이터, 어느 피쳐, 어느 파라미터로 훈련한 모델인가”를 기록하지 않고는 어떤 결과도 재현할 수 없다. 9장은 이 기록의 체계 — 실험 추적과 모델 레지스트리(MLflow)를 다룬다. 이 장의 피쳐(전일 민원 건수)가 실제 훈련의 입력이 되고, 7장의 재현성 원칙(같은 입력→같은 산출물)이 “같은 실험 설정→같은 모델”로 확장된다.

참고문헌

Feast. (n.d.). *Feast Documentation*. <https://docs.feast.dev/> (직접 선례 — 실습 도구)
Feast. (n.d.). *Concepts*. <https://docs.feast.dev/getting-started/concepts> (직접 선례)
Feast. (n.d.). *Feature view*. <https://docs.feast.dev/getting-started/concepts/feature-view> (직접 선례)

Feast. (n.d.). *Point-in-time joins*. <https://docs.feast.dev/getting-started/concepts/point-in-time-joins> (직접 선례)

Feast. (n.d.). *Architecture Overview*. <https://docs.feast.dev/getting-started/architecture/overview> (직접 선례)

Google. (n.d.). *Rules of Machine Learning: Best Practices for ML Engineering*. <https://developers.google.com/machine-learning/guides/rules-of-ml> (방법론 차용 — 훈련-서빙 스큐 정의, 민간·일반 대상. 1장 공통 인용)

Tecton. (n.d.). *What Is a Feature Store?*. <https://www.tecton.ai/blog/what-is-a-feature-store/> (방법론 차용 — 상용 벤더 기술 문서. 피쳐 스토어 기원·정의·훈련-서빙 스큐·point-in-time)

Hopsworks. (n.d.). *Feature Store: The Definitive Guide*. <https://www.hopsworks.ai/dictionary/feature-store> (배경 자료 — online/offline·최신성·point-in-time 개념)